

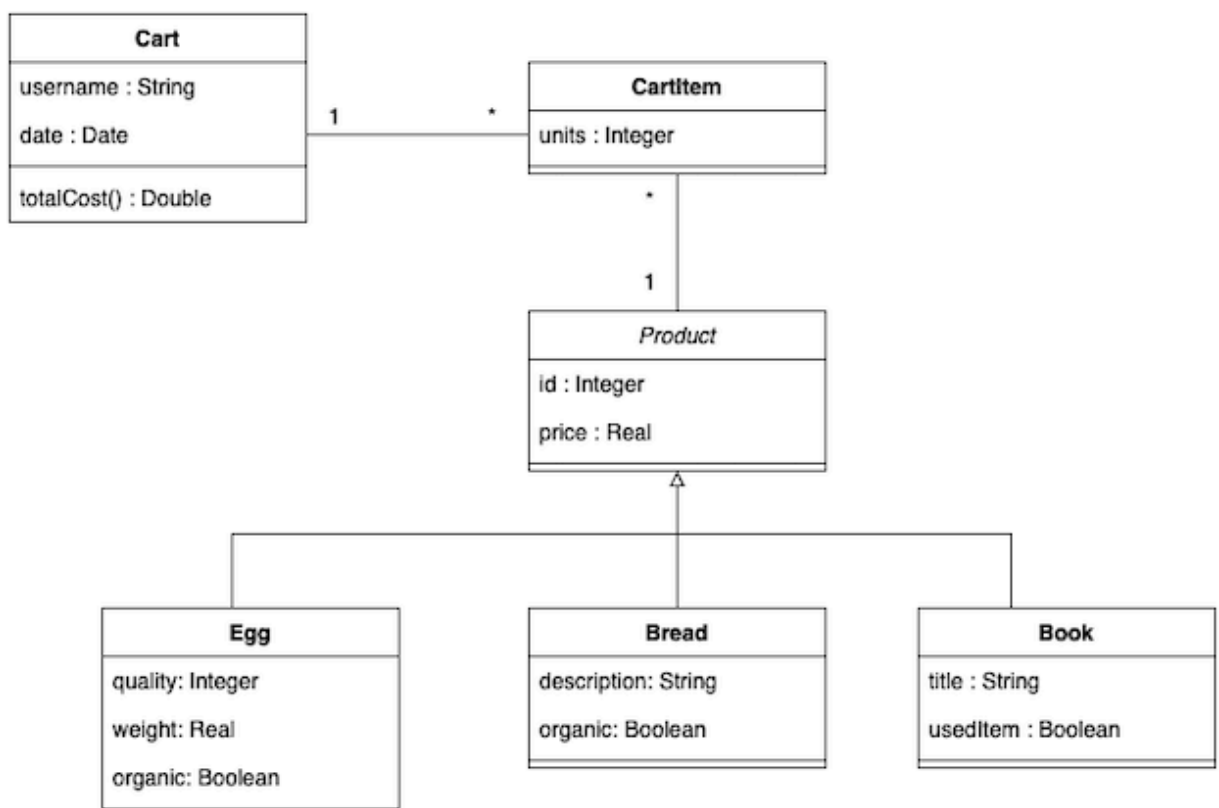
Análisis y Diseño con Patrones

PEC 1: Principios de Diseño y Patrones de Análisis

Pregunta 1 (12,5 puntos)

Enunciado

"Breggs and Books" es una pequeña librería de barrio especial. Originalmente, vendía únicamente libros, pero en los últimos años también vende huevos y pan de pequeños productores. Su éxito ha ido en aumento y han decidido dar el salto a tener una tienda online. En el modelado de proyecto, el equipo de ingeniería propone el siguiente diagrama para ilustrar el modelo de un carro de la compra.



En este modelo *Cart* representa un carro de compra, el cual tiene *username* para referirse al usuario que realiza la compra, una fecha en la que se realiza la compra (*date*) y una serie de *CartItems*. Cada uno de estos *CartItems* tiene el atributo *units* para representar el número de unidades del producto a comprar y el propio producto (*Product*), el cual puede ser *Bread*, *Egg* o *Book*. Los atributos de *Egg* son la calidad del huevo en valor numérico (*quality*), el peso del huevo (*weight*) y un booleano para informar de si es ecológico (*organic*). Por otro lado, los atributos de *Bread* son la descripción del tipo de pan (*description*) y un booleano para informar de si es ecológico (*organic*). Finalmente, los atributos de *Book* son el título (*title*) y un booleano que sirve para informar de si se trata de un producto usado (*usedItem*).

Además hay un par de consideraciones importantes:

- El método *totalCost()* sirve para calcular el precio total del carro de compra.
- En caso de que el carro de la compra tenga libros, estos tendrán un 10% de descuento.

El pseudocódigo relativo al método *totalCost()* es el siguiente:

```
public class Cart {  
    private String username;  
    private Date date;  
    private List<CartItem> cartItems;  
  
    public Real totalCost(){  
        Real total = 0.0  
        for (CartItem cartItem : cartItems) {  
            if(typeof(cartItem.product) == Book)  
                total = total + (cartItem.product.price * cartItem.units) * 0.9;  
            else  
                total = total + (cartItem.product.price * cartItem.units);  
        }  
        return total;  
    }  
}
```

Se pide:

- a) (5 puntos) Tanto en el diagrama como en el pseudocódigo de la operación *totalCost()* se están produciendo una serie de violaciones a los principios de diseño. Explica detalladamente qué principios se están incumpliendo y qué desventajas presenta un modelado con estas violaciones.
- b) (7,5 puntos) Realiza las correcciones necesarias para implementar correctamente el modelado sin violar ningún principio de diseño. Es necesario incluir tanto el diagrama de clases como todo el pseudocódigo de las clases involucradas.

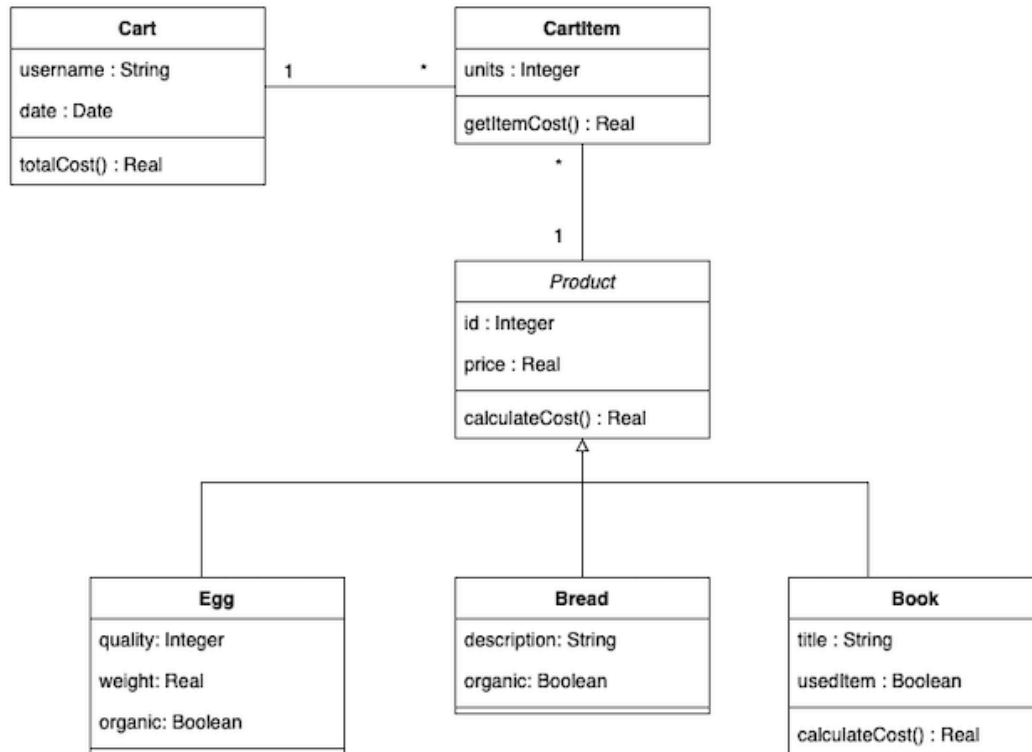
Solución

- a) En primer lugar, no se satisface la Ley de Demeter. Esta ley establece que un objeto debería tener conocimiento limitado sobre otros objetos, y que no debería acceder a los componentes internos de otros objetos. En el contexto del ejercicio, *Cart* está accediendo a muy bajo nivel, navegando directamente a los detalles de cada uno de los ítems del carro para calcular su precio. Esto es un error, ya que *Cart* sólo debería conocer los detalles de los objetos que posee, en lugar de acceder a sus componentes internos. En esta ocasión está navegando hasta *Product*, clase con la que no está directamente relacionada. Cumplir este principio mejora la modularidad y reduce el acoplamiento en el código, facilitando su mantenimiento y evolución.

Por otro lado, se está violando el principio de abierto-cerrado. Este principio establece que una clase debe estar abierta para su extensión pero cerrada para su modificación. En el ejemplo, el Carro de Compras está comprobando si un producto es de tipo *Book* para aplicar un descuento específico. Si en el futuro el sistema creciese y se decidiera agregar un nuevo tipo de producto o modificar los descuentos existentes, sería difícil hacerlo debido al acoplamiento existente en el código. Esto haría que el sistema no cumpla con el principio de estar abierto a la extensión pero cerrado a la modificación, lo que dificultará su mantenimiento y escalabilidad.

Finalmente, también es necesario mencionar que *Cart* está altamente acoplada a *Product* y a *Book*. La funcionalidad de *Cart* depende en gran medida de cómo están implementadas *Product* y *Book*, lo que es un error. Recordemos que alto acoplamiento indica una fuerte dependencia entre las clases, lo cual puede hacer que el código sea más difícil de entender, mantener y modificar.

b) El diagrama de clase que se propone como solución es el siguiente



Primeramente, para evitar la violación de la Ley de Demeter, lo que hemos hecho ha sido eliminar todo el cálculo que se hacía en *Cart*, haciendo que sea *CartItem* quien calcule cada coste por separado, debido a que conoce tanto el producto como la cantidad del mismo. Por otro lado, para eliminar la violación del principio de abierto-cerrado, hemos hecho que cada uno de los productos calcule su propio coste. De este modo, si alguno tiene algún descuento o comportamiento especial por su naturaleza, se puede hacer de un modo fácil y no tenemos el sistema acoplado, sino que es flexible ante la extensión. Además, con estos cambios también hemos desacoplado *Cart* de *Product* y *Book*, ya que ahora la funcionalidad de *Cart* no está directamente relacionada a cómo estén implementados tanto *Product* como *Book*.

El pseudocódigo es el siguiente:

```

public class Cart {
    private String username;
    private Date date;
    private List<CartItem> cartItems;

```

```
public Real totalCost(){
    Real total = 0.0
    for (CartItem cartItem : cartItems){
        total = total + cartItem.getItemCost();
    }
    return total;
}

public class CartItem {
    private Integer units;
    private Product product;

    public Real getItemCost(){
        return product.calculateCost() * units;
    }
}

public abstract class Product {
    private Integer id;
    protected Real price;

    public Real calculateCost(){
        return this.price;
    }
}

public class Bread extends Product {
    private String type;
    private Boolean organic;
}

public class Egg extends Product {
    private Integer quality;
    private Real weight;
    private Boolean organic;
}

public class Book extends Product {
    private String description;
    private Boolean usedItem;

    public Real calculateCost(){
        return this.price * 0.9;
    }
}
```

Pregunta 2 (12,5 puntos)

Enunciado

Los principios de diseño desempeñan un papel crucial en el desarrollo de software, ya que definen la calidad de su estructura. Uno de estos principios, conocido como el principio de segregación de interfaces, es esencial para una arquitectura sólida. Investiga en profundidad sobre este principio, aprovechando los recursos disponibles en la asignatura y las referencias proporcionadas en el aula.

Una vez que hayas comprendido completamente el principio de segregación de interfaces, se requiere que proporciones un ejemplo que muestre cómo este principio puede ser violado y que lo expliques con detalle. Este ejemplo deberá incluir un diagrama de clases y restricciones de integridad pertinentes (en caso de que las haya). Además, explica por qué se produce dicha violación del principio.

¿Cuáles crees que serían las consecuencias de no respetar este principio en el diseño de software?

Nota: podéis modificar el diagrama de la pregunta 1 cómo creáis necesario (añadiendo clases nuevas, atributos, métodos, etc..) para que se viole el principio de segregación de interfaces.

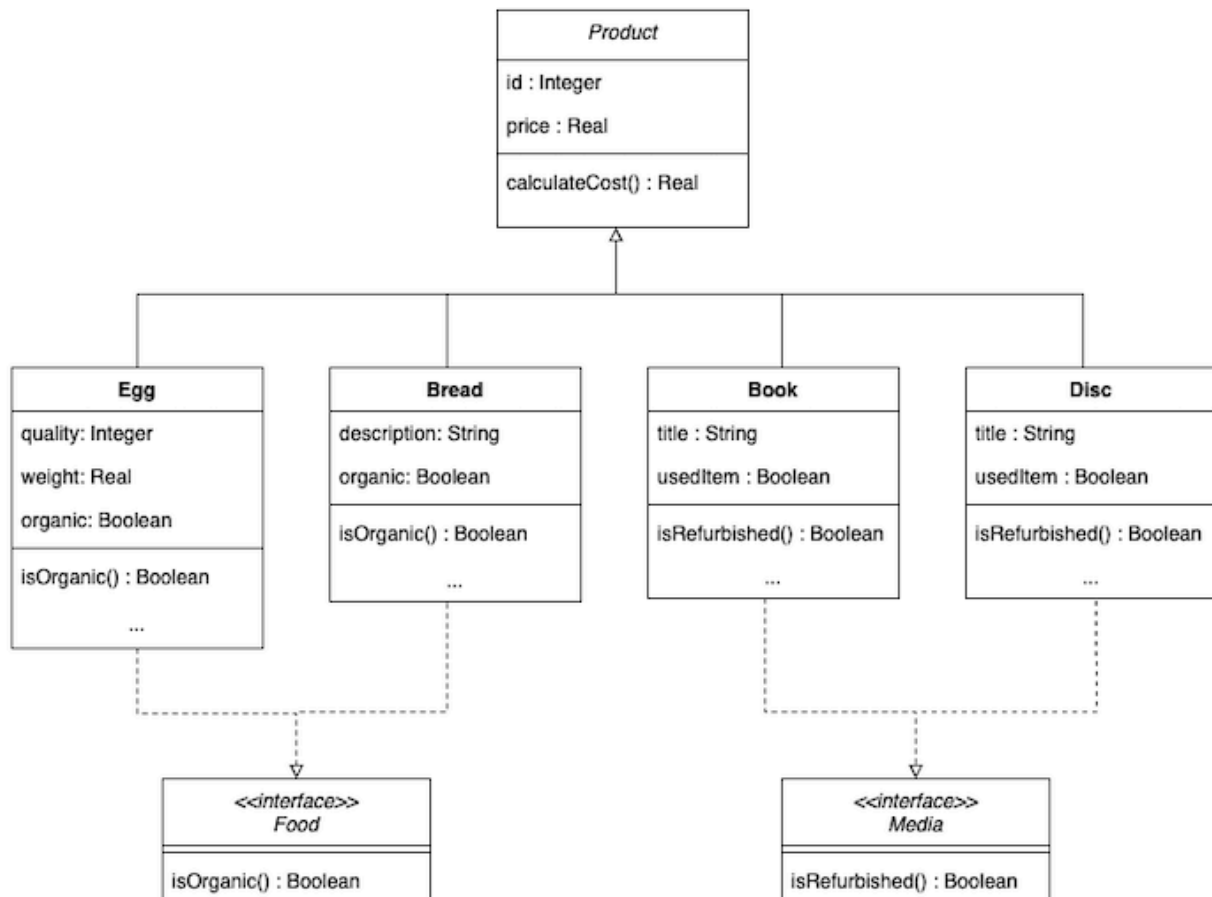
Solución

En primer lugar, tenemos que considerar que el principio de segregación de interfaces establece que los clientes no deben verse obligados a depender de interfaces que no utilizan. No llevarlo a cabo puede resultar en acoplamiento excesivo entre componentes, dificultando la mantenibilidad y escalabilidad del sistema.

Basándonos en el ejemplo del apartado 1 vamos a considerar que además de *Egg*, *Bread* y *Book*, también comienzan a vender *Disc*. Podemos ver que hay una doble naturaleza, por un lado alimentos y por otro productos. Además, vemos que tanto *Egg* como *Bread* tienen como propiedad el hecho de ser *organic*, mientras que *Book* y *Disc* pueden compartir ser *refurbished* o *used item*.

En un primer momento, podríamos pensar en definir un método *isOrganic()* y otro *isRefurbished()* en *Product*. Sin embargo, esto sería incorrecto, ya que los alimentos no son *refurbished* y los libros

no son *organic*. De esto modo, la solución que nos permite no violar el principio de segregación puede ser la siguiente.



Como vemos, hemos creado dos interfaces que nos permiten abstraer de un modo sencillo ese comportamiento, permitiendo al sistema ser más extensible y estando menos acoplado

Pregunta 3 (15 puntos)

Enunciado

Durante la temporada de verano, los hoteles de la cadena Verano S.A. experimentan un aumento considerable en la demanda de personal debido al incremento de turistas. Para hacer frente a esta situación, la empresa ha decidido implementar una estrategia flexible que involucra la movilidad de sus empleados entre distintos establecimientos según las necesidades del momento. Es por ello que se plantea la necesidad de desarrollar un sistema eficiente de registro que permita mantener un seguimiento detallado de la ubicación actual y pasada de cada trabajador.

Es importante destacar que, dada la naturaleza dinámica de esta práctica, un empleado puede ser asignado a trabajar en un hotel durante un período determinado, luego trasladarse a otro durante otro lapso de tiempo y, eventualmente, regresar al lugar de origen. Sin embargo, es fundamental garantizar que en todo momento los empleados estén activos y desempeñando sus labores dentro de la red hotelera de Verano S.A.

Se pide:

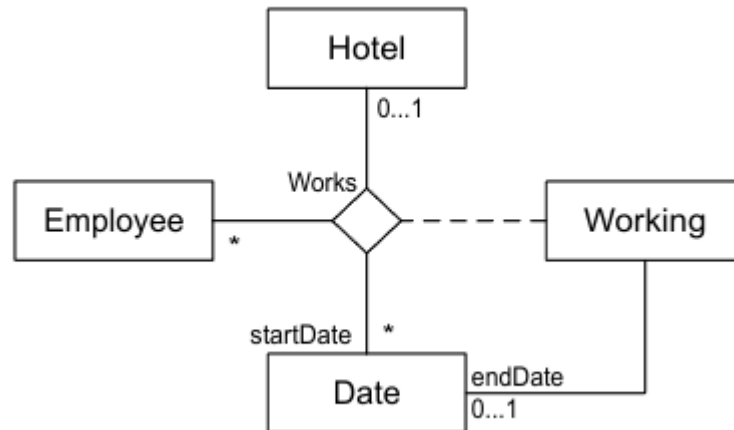
- a) (5 puntos) Indicad qué patrones de análisis tenemos que tener en cuenta para implementar este requisito y justificad brevemente por qué.
- b) (10 puntos) Proponed un diagrama estático de análisis donde se apliquen los patrones de análisis identificados en los requisitos descritos y las restricciones de integridad.

Nota: Si hace falta, podéis hacer alguna variación sobre los patrones tal y como están explicados en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

- a) En esta situación, podemos usar un método llamado "Asociación histórica", que nos ayuda a llevar un registro de los tiempos en que los empleados han trabajado en cada hotel. Es como mantener un historial de dónde han estado y cuándo. Esto es útil para asegurarnos de tener un seguimiento de la experiencia laboral de cada empleado en diferentes lugares.

b)



Restricciones de integridad

- Cada empleado debe tener al menos un período de trabajo sin una fecha de finalización definida, lo que indica que el empleado está actualmente trabajando.
- Para cada período en el que un empleado haya trabajado en un hotel, se debe registrar un nuevo periodo en el que el empleado trabaje en otro hotel, comenzando al día siguiente de la fecha de finalización del período anterior.
- No se permiten períodos superpuestos en los que un empleado trabaje en más de un hotel simultáneamente.

Pregunta 4 (10 puntos)

Enunciado

Una empresa que fabrica termómetros para medir la temperatura ambiente quiere desarrollar un sistema para guardar la información de los diferentes tipos de termómetros. Un tipo de termómetro tiene un nombre que lo identifica, una descripción y la escala en la que mide la temperatura. Por ejemplo, el termómetro con nombre TEMAMB, descripción “Termómetro digital para registro de temperatura ambiente” mide la temperatura en la escala Celsius. Actualmente, la empresa fabrica termómetros que miden la temperatura en Celsius y Fahrenheit. Cada tipo de termómetro, en función de la sensibilidad de sus sensores, puede medir la temperatura entre un valor mínimo y un valor máximo. Por ejemplo el termómetro TEMAMB puede medir temperaturas entre -10°C y 50°C.

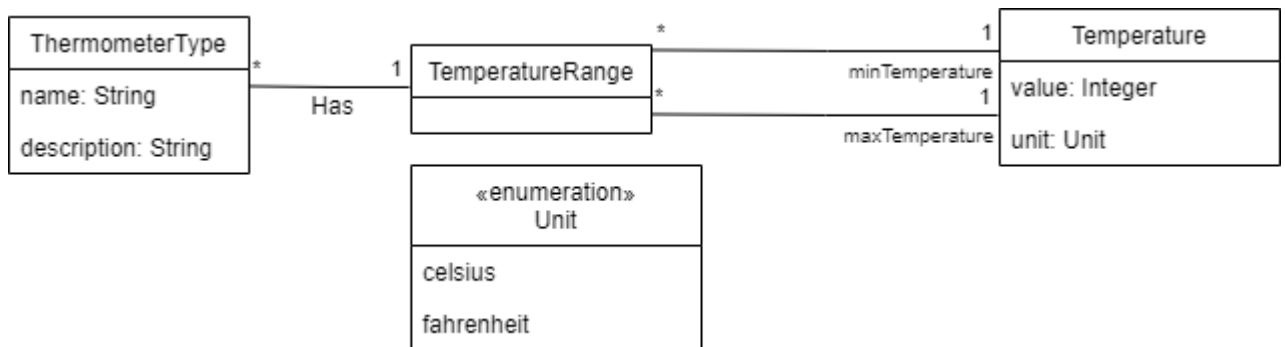
Se pide:

- a) (2.5 puntos) Indicad qué patrones de análisis tenemos que tener en cuenta para diseñar este sistema y justificad brevemente por qué.
- b) (7.5 puntos) Proponed un diagrama estático de análisis donde se apliquen los patrones de análisis identificados y las restricciones de integridad.

Nota: Si hace falta, podéis hacer alguna variación sobre los patrones tal y como están explicados en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

- a) Los patrones de análisis más adecuados son: 1) el patrón Rango para definir el rango de temperaturas que mide un tipo de termómetro, 2) el patrón Cantidad para representar el valor de las temperaturas y la unidad en la que se miden.
- b)



Restricciones de integridad

- El identificador del tipo de termómetro es su nombre.
- El identificador de Temperatura es el valor y la unidad.
- El valor mínimo de la temperatura de un tipo de termómetro tiene que ser inferior a su valor máximo.